

OUT-OF-BOUNDS

C++ Circuit Simulator

Project Documentation

[Mathematics](#) • [Logic](#) • [MNA](#) • [Numerical Solver](#) • [Workflow](#) • [Demonstration](#)

Repository	github.com/imbharat2510/Out-of-Bounds
Language	C++17
Core method	Modified Nodal Analysis
Analysis	DC and AC frequency sweep

Developed as a collaborative project to implement and understand the fundamentals of circuit simulation and Modified Nodal Analysis.

1. Project Overview

Out-of-Bounds is an interactive command-line circuit simulator written in C++. The user specifies the number of nodes, with node 0 as ground, then adds supported components through a menu. The program finally chooses DC or AC analysis and solves the circuit mathematically.

The repository contains separate circuit, component, analysis and numerical-solver modules. The public repository currently contains include/, src/, a Makefile and the compiled executable.

Core features

- Interactive circuit construction.
- Resistors, inductors, capacitors and voltage sources.
- Automatic component naming.
- Input validation.
- DC node-voltage analysis.
- AC frequency sweep with magnitude and phase.
- Complex-valued Modified Nodal Analysis.
- Gaussian elimination with partial pivoting.

Main mathematical pipeline

Circuit topology + component values → impedances/admittances → MNA matrix A and vector b → solve $Ax=b$ → node voltages → displayed results.

2. Software Architecture

Module	Role
main.cpp	Input, validation, component creation, analysis selection and output.
Circuit.h	Stores node count, ground node and component objects.
Component.h	Base component interface plus R, L, C and voltage-source implementations.
MNASimulator.cpp	MNA matrix construction, component stamping and Gaussian elimination.
DCAalysis.cpp	Calls the simulator once at 0 Hz.
ACAnalysis.cpp	Calls the simulator for each frequency in the sweep.
Linalg.h	Defines complex scalar, matrix and vector types.
Makefile	Builds, runs and cleans the program.

The circuit owns components using `std::unique_ptr`. The component classes expose common methods such as `getImpedance()`, `getValue()`, `getType()`, `getNodeA()` and `getNodeB()`.

The numerical layer uses `std::complex<double>` for matrix entries, allowing the same solver to handle real DC equations and complex AC equations.

Build command

```
g++ -std=c++17 -Wall -O2 -Iinclude src/main.cpp src/ACAnalysis.cpp src/DCAalysis.cpp  
src/MNASimulator.cpp -o circuit_sim.exe
```

3. Component Mathematics

AC analysis uses phasors. For angular frequency $\omega = 2\pi f$, each passive element is represented by a complex impedance. The MNA code then uses the reciprocal admittance $Y = 1/Z$.

Element	Impedance Z	Admittance Y
Resistor	R	$1/R$
Inductor	$j\omega L$	$1/(j\omega L)$
Capacitor	$1/(j\omega C)$	$j\omega C$

For $f = 0$ the capacitor is an open circuit. The inductor is treated as an ideal short in DC using a zero-voltage branch equation. An ideal voltage source also requires an additional branch-current unknown.

DC interpretation

- Resistor: remains a conductance $1/R$.
- Capacitor: open circuit, so no DC current.
- Inductor: short circuit, represented through an additional zero-voltage branch equation.
- Voltage source: requires an additional branch-current unknown in MNA.

4. Modified Nodal Analysis

Ordinary nodal analysis works naturally for admittances. An ideal voltage source has no direct current-voltage relation of the form $I = YV$, so MNA introduces the current through that branch as another unknown.

For N non-ground nodes and M branch-current unknowns:

$$x = [V_1, V_2, \dots, V_N, I_1, I_2, \dots, I_M]^T$$

$$\begin{bmatrix} G & B \\ B^T & 0 \end{bmatrix} \begin{bmatrix} V \\ I \end{bmatrix} = \begin{bmatrix} b_{\text{node}} \\ b_{\text{branch}} \end{bmatrix}$$

For a passive element of admittance Y between nodes a and b :

$$\begin{aligned} A_{aa} &+= Y & A_{bb} &+= Y \\ A_{ab} &-= Y & A_{ba} &-= Y \end{aligned}$$

For a voltage-defined branch k :

$$\begin{aligned} A_{a,k} &+= 1, & A_{b,k} &-= 1 \\ A_{k,a} &+= 1, & A_{k,b} &-= 1 \\ b_k &= V_s \end{aligned}$$

The matrix dimension is $N+M$. Ground is not an unknown; node 0 is fixed at 0 V.

5. Worked Example From the Provided Run

The demonstration uses $V1 = 10\text{ V}$ at 1000 Hz , $R1 = 50\ \Omega$ between nodes 1-2, $L1 = 0.01\text{ H}$ between nodes 2-3, and $C1 = 2.53\ \mu\text{F}$ between node 3 and ground.

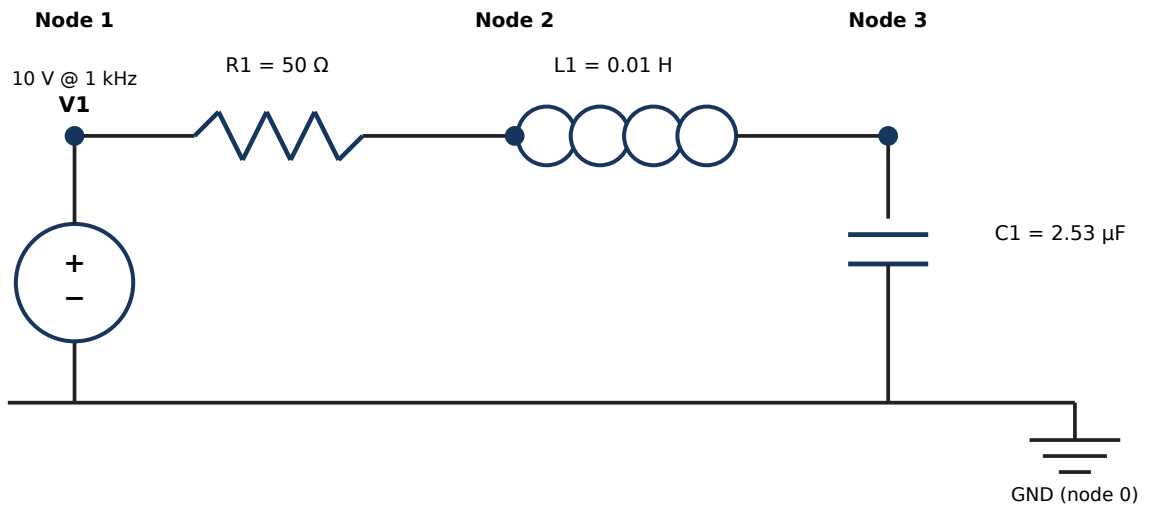


Figure 1. Improved schematic of the circuit used in the demonstration.

Element	Nodes	Value
V1	1-0	10 V, 1000 Hz
R1	1-2	$50\ \Omega$
L1	2-3	0.01 H
C1	3-0	$2.53 \times 10^{-6}\text{ F}$

At $f = 1000\text{ Hz}$: $\omega = 6283.185\text{ rad/s}$.

$$Y_R = 0.02\text{ S} \quad Y_L = -j0.0159155\text{ S} \quad Y_C = j0.0000190355\text{ S}$$

6. Exact Matrix for the Demonstration

There are three non-ground nodes and one voltage-source branch current, so the unknown vector has four entries:

$$\begin{aligned} \mathbf{x} &= [V_1, V_2, V_3, I_{V1}]^T \\ &\begin{bmatrix} 0.0200 & -0.0200 & 0 & 1 \\ -0.0200 & 0.0200 - j0.0159155 & j0.0159155 & 0 \\ 0 & j0.0159155 & -j0.0000190355 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \\ \mathbf{b} &= [0, 0, 0, 10]^T \end{aligned}$$

Row-by-row meaning

- Row 1: KCL at node 1; R1 and V1 contribute.
- Row 2: KCL at node 2; R1 and L1 contribute.
- Row 3: KCL at node 3; L1 and C1 contribute.
- Row 4: ideal source constraint $V1 - V0 = 10 \text{ V}$.

Solution

Node	Magnitude	Phase
0	0.0000 V	0.0000°
1	10.0000 V	0.0000°
2	0.0150 V	-89.9138°
3	12.5814 V	-89.9138°

7. Linear-System Solver

After matrix construction, solveLinear() performs Gaussian elimination with partial pivoting on complex matrix entries.

1. Choose pivot

For each column c , search rows $c \dots n-1$ for the largest $|A[r][c]|$. Swap that row into the pivot position. A pivot magnitude below 10^{-12} is treated as singular.

2. Eliminate below pivot

$$\begin{aligned} \text{factor} &= A_{r,c} / A_{c,c} \\ A_{r,*} &\leftarrow A_{r,*} - \text{factor} \cdot A_{c,*} \\ b_r &\leftarrow b_r - \text{factor} \cdot b_c \end{aligned}$$

3. Back substitution

$$x_i = (b_i - \sum_{j=i+1}^{n-1} A_{ij} x_j) / A_{ii}$$

4. Extract node voltages

The solution vector contains node voltages and branch currents; the returned result contains the node-voltage entries, with node 0 explicitly set to 0 V.

Complexity

For an $S \times S$ dense matrix, Gaussian elimination takes $O(S^3)$ time and $O(S^2)$ memory, where $S=N+M$.

8. Detailed Program Workflow

8.1 Startup

- Read number of nodes.
- Require at least two nodes.
- Create Circuit(numNodes), with node 0 as ground.

8.2 Circuit builder

The menu accepts component choices. Node values are validated, component values must be valid and positive where appropriate, and components are stored in the Circuit object.

8.3 Selecting analysis

The program scans voltage sources. If no source has frequency > 0 , it executes DC analysis at 0 Hz. Otherwise it requests start frequency, end frequency and step.

8.4 DC workflow

DCAnalysis::run() calls the simulator at 0 Hz and prints the resulting node voltages.

8.5 AC workflow

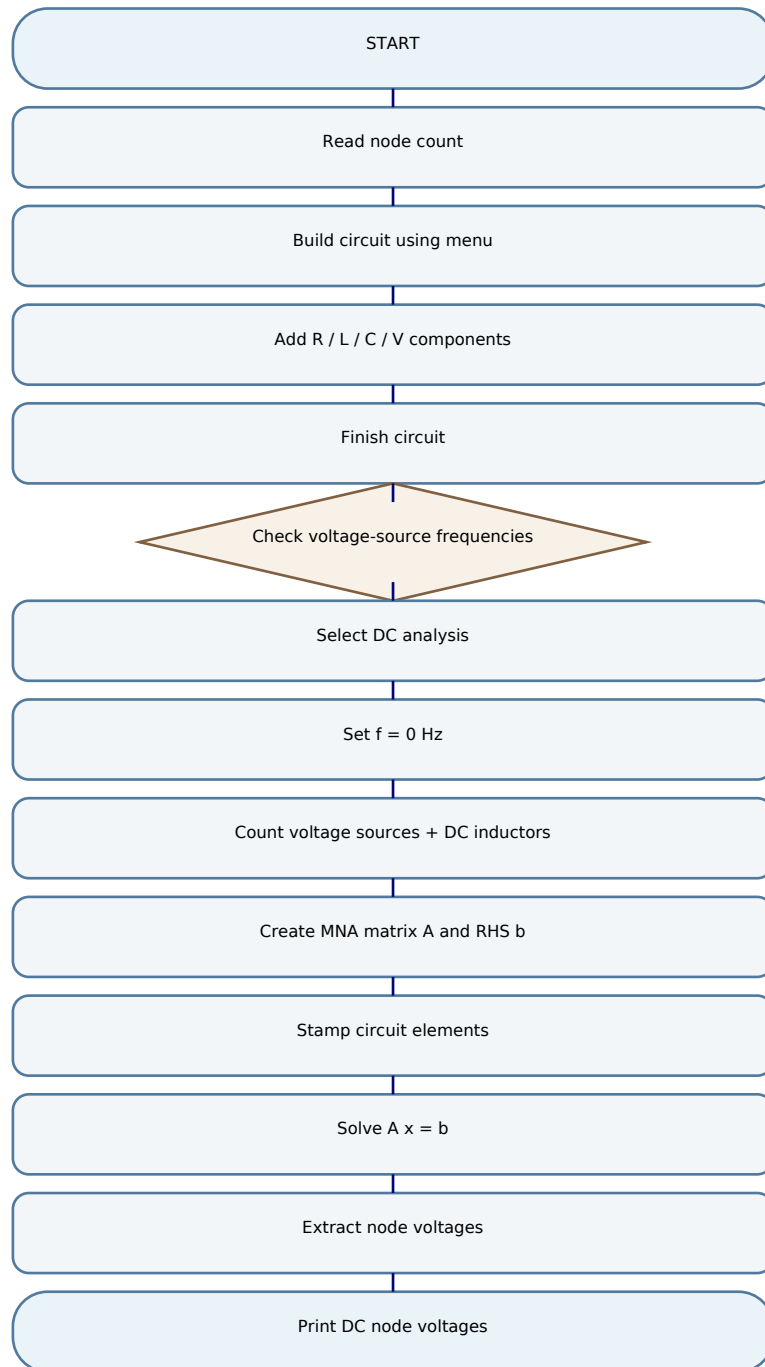
ACAnalysis::run() loops over the frequency range, calls the same MNA solver at each frequency, and prints each node voltage as magnitude and phase.

8.6 Error handling

A singular MNA matrix causes an analysis failure. Typical causes include floating nodes, missing ground reference and invalid voltage-source loops.

9. DC Analysis Code Flowchart

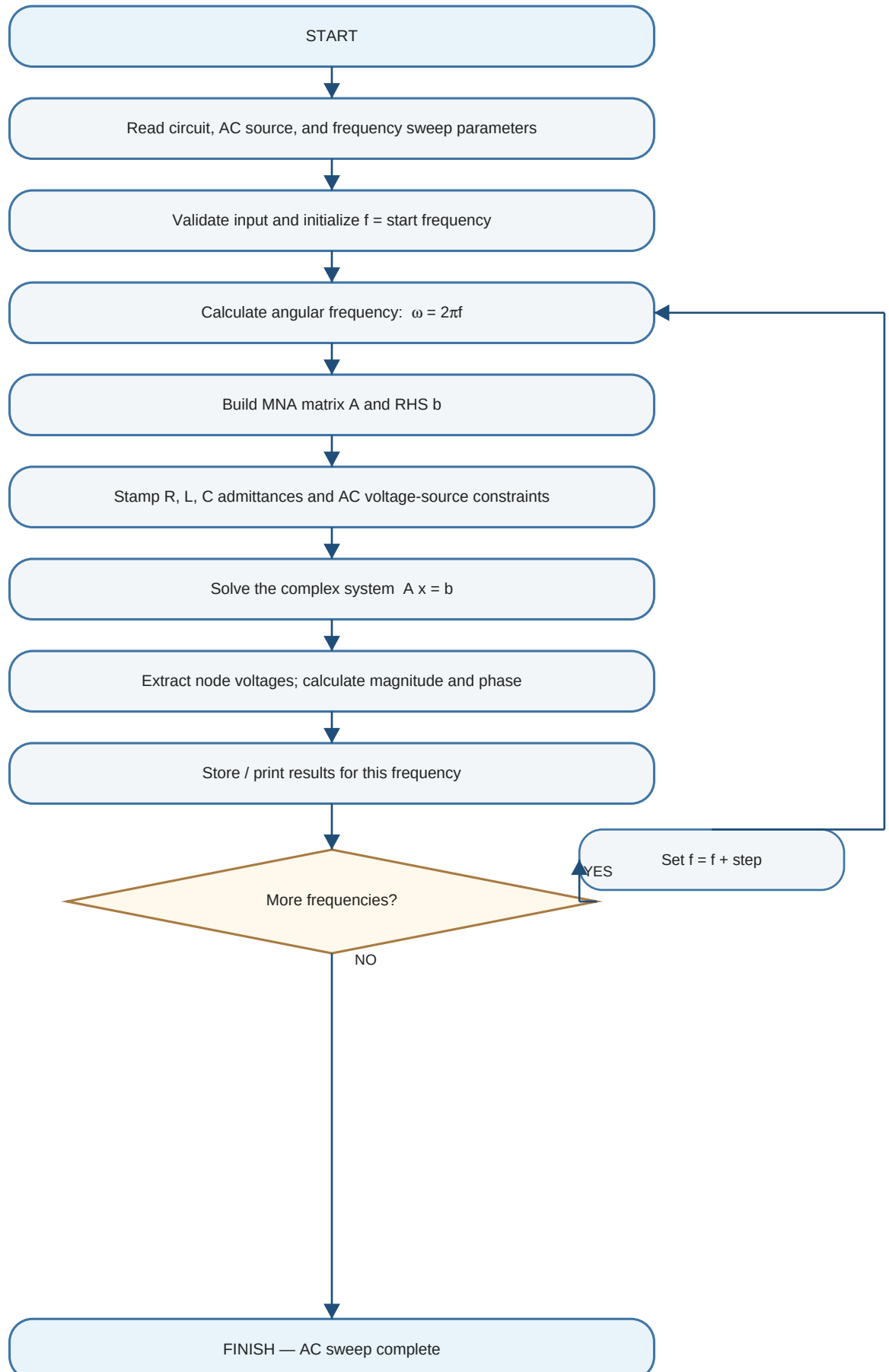
The DC path is selected when the circuit has no AC voltage source. The MNA solver is called once at $f = 0$ Hz.



Flowchart of the program execution path.

10. AC Analysis Code Flowchart

The AC path performs MNA at each frequency in the requested sweep and records the node-voltage response.



11. Demonstration Screenshots

The following images were provided with the project request.

```
PS C:\Projects\CPP\Out-of-Bounds> mingw32-make run
.\circuit sim.exe
Number of nodes (including ground node 0): 4

Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
5. List components
0. Finish
Choose an option: 1

Adding resistor R1
Node A: 1
Node B: 2
Resistance in ohms: 50
Added R1.

Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
5. List components
0. Finish
Choose an option: 2

Adding inductor L1
Node A: 2
Node B: 3
Inductance in henries: 0.01
Added L1.

Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
```

Figure 7. Adding R1 and L1 through the circuit builder.

```
Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
5. List components
0. Finish
Choose an option: 3

Adding capacitor C1
Node A: 3
Node B: 0
Capacitance in farads: 0.00000253
Added C1.

Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
5. List components
0. Finish
Choose an option: 4

Adding voltage source V1
Node A: 1
Node B: 0
Voltage in volts: 10
Source frequency in Hz (0 for DC): 1000
Added V1.

Circuit Builder
1. Add resistor
2. Add inductor
3. Add capacitor
4. Add voltage source
5. List components
```

Figure 8. Adding C1 and the AC voltage source.

$C1 = 0.00000253 \text{ F}$ and $V1 = 10 \text{ V}$ at 1000 Hz .

```
Final circuit:
Circuit: 4 nodes (node 0 = ground)
R R1 nodes(1,2) value=50 ohms
L L1 nodes(2,3) value=0.01 H
C C1 nodes(3,0) value=2.53e-06 F
V V1 nodes(1,0) value=10 V frequency=1000 Hz

AC sweep start frequency in Hz: 1000
AC sweep end frequency in Hz: 1000
AC sweep step in Hz: 1

AC frequency sweep (node voltage magnitude and phase):
Frequency: 1000.0000 Hz
V(node 0) = 0.0000 V, phase = 0.0000 deg
V(node 1) = 10.0000 V, phase = 0.0000 deg
V(node 2) = 0.0150 V, phase = -89.9138 deg
V(node 3) = 12.5814 V, phase = -89.9138 deg
PS C:\Projects\CPP\Out-of-Bounds> 
```

Figure 9. Final circuit listing and AC sweep output.

The sweep uses start = end = 1000 Hz and step = 1 Hz, so one frequency point is reported.

12. Contributors and Repository

Repository: <https://github.com/imbharat2510/Out-of-Bounds>

The project was developed collaboratively, with responsibilities divided across circuit-analysis implementation, numerical logic, input handling, testing and documentation.

Contributor	Roll Number	Contribution
Aarya	IE2025004	AC and DC analysis, output handling, testing
Aditya	BE2025001	Logic, matrix solving, documentation
Bharat	BE2025007	Logic, forming equations and creating matrix, debugging
Mayank	BE2025018	Input handling and formatting, menu structure

Developed as a collaborative project to implement and understand the fundamentals of circuit simulation and Modified Nodal Analysis.

Repository contents referenced in this document

main.cpp, MNASimulator.cpp, DCAnalysis.cpp, ACAnalysis.cpp, Component.h, Circuit.h, Linalg.h and the Makefile.